

Tema 01 – Arquitecturas y Herramientas de Programación Web

2º DAW – Desarrollo Web Entorno Servidor

Profesor Juan Carlos Moreno

CURSO 2026/2027

Tabla de Contenido

1	Arquitecturas y Herramientas de Programación Web	3
1.1	Modelo de Programación en Entorno Cliente/Servidor	3
1.1.1	Capas de un sistema web	4
1.1.2	Tipos de arquitecturas cliente/servidor	5
1.2	Páginas Web Estáticas y Dinámicas	6
1.3	Lenguajes de Programación en Entorno Servidor	7
1.3.1	Lenguajes de Scripting	8
1.3.2	Aplicaciones CGI y Derivados	9
1.3.3	Aplicaciones Híbridas de código repartido (Code-Behind).....	10
1.3.4	Ventajas e inconvenientes de cada tipo de lenguaje	10
1.4	Integración con los Servidores Web	11
1.5	Herramientas de Programación.....	12
1.5.1	Marcadores de Texto.....	13
1.5.2	Herramientas Genéricas.....	14
1.5.3	Herramientas Específicas	14
1.6	Servicios Web.....	15
1.6.1	Plataforma XAMPP	15
1.6.2	MySQL Workbench.....	15

1 Arquitecturas y Herramientas de Programación Web

En este primer capítulo presentamos los conceptos necesarios para comprender el contexto del desarrollo de aplicaciones y sistemas de información web. Para ello, introducimos los modelos de programación más comunes en la Web, así como los diferentes lenguajes y tecnologías de programación, del lado del servidor, aplicables en este tipo de entornos. Por último, haremos un recorrido por algunas de las herramientas que el desarrollador tiene a su alcance para sacarle el máximo partido a la programación web.

Objetivos:

- Caracterizar y diferenciar los modelos de ejecución de código en un entorno cliente/servidor.
- Conocer los mecanismos de ejecución de código en función de las diferentes tecnologías web.
- Reconocer las ventajas de la generación dinámica de páginas web.
- Identificar y caracterizar los principales lenguajes y tecnologías de programación en entorno servidor.
- Conocer las distintas herramientas de programación web para lenguajes de servidor.

1.1 Modelo de Programación en Entorno Cliente/Servidor

La World Wide Web (o la Web, como se conoce comúnmente) representa un universo de información accesible globalmente a través de internet. Está formada por un conjunto de recursos interconectados que conforman el conocimiento humano actual. El funcionamiento de la Web es posible debido a la coexistencia de una serie de componentes software y hardware. Estos elementos abarcan desde los componentes físicos de internet (hubs, repetidores, puentes, pasarelas, encaminadores, etc.) y los protocolos de comunicaciones (TCP, IP, HTTP, FTP, SMTP, etc.) hasta la utilización del sistema de nombres de dominio (DNS) para la búsqueda y recuperación de recursos o la utilización de software específico para proveer y consumir dichos recursos.

En este contexto, el desarrollo en entornos web debe tener en cuenta la distribución de los elementos y la función que tiene cada uno de ellos. La configuración arquitectónica más habitual se basa en el modelo denominado **Cliente/ Servidor**, basado en la idea de servicio, en el que el cliente es un componente consumidor de servicios y el servidor es un proceso proveedor de servicios. Además, esta relación está robustamente cimentada en el intercambio de mensajes como el único elemento de acoplamiento entre ambos.

El agente que solicita la información se denomina **cliente**, mientras que el componente software que responde a esa solicitud es el que se conoce como **servidor**. En un proceso habitual el cliente es el que inicia el intercambio de información, solicitando datos al servidor, que responde enviando uno o más flujos de datos al cliente. Además de la transferencia de datos real, este intercambio puede requerir información adicional, como la autenticación del usuario o la identificación del archivo de datos que vayamos a transferir.

La Figura 1.1 muestra este proceso genérico.

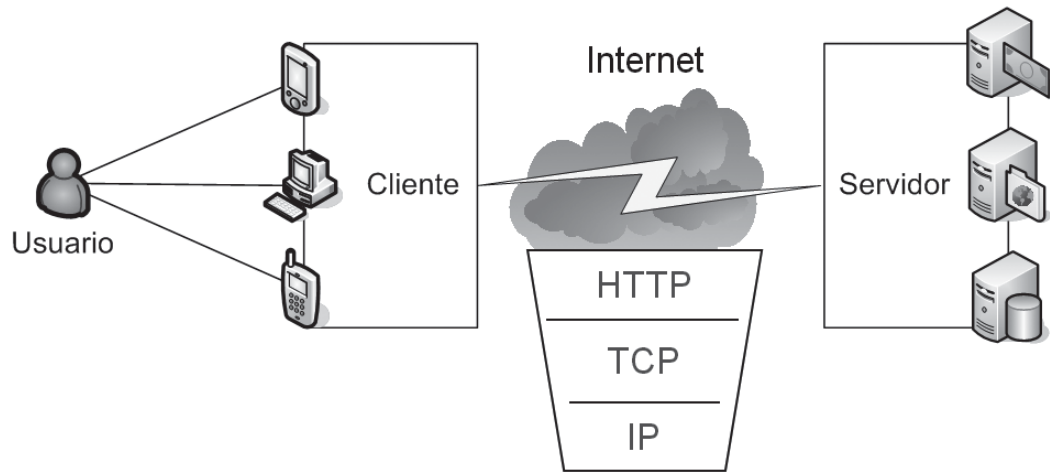


Ilustración 1-1. Modelo Cliente/Servidor

Ejemplo de flujo de trabajo cliente/servidor:

1. El cliente (navegador web), al hacer clic en un enlace o escribir una URL, envía una solicitud HTTP al servidor.
2. El servidor web, al recibir la solicitud, la procesa.
3. El servidor envía una respuesta HTTP que contiene el recurso solicitado (por ejemplo, un archivo HTML, una imagen o datos JSON).
4. El cliente (navegador) recibe la respuesta y la renderiza para que el usuario pueda ver la página.

1.1.1 Capas de un sistema web

Las funcionalidades en los entornos cliente/servidor de la Web suelen estar agrupadas en diferentes capas, cada una centrada en la gestión de un aspecto determinado del sistema web. Si bien es posible encontrarse con divisiones más o menos especializadas, tradicionalmente se identifican tres tipos de capas fundamentales: **capa de presentación (Frontend)**, **capa de la lógica de negocio (Backend)** y **capa de persistencia (base de datos)**. Los modelos arquitectónicos de programación se pueden clasificar en función de en qué lugar, si en el cliente o en el servidor, se sitúan cada una de estas capas. La decisión de dónde se sitúa cada una de estas capas dependerá del entorno de ejecución y, por tanto, de las tecnologías y lenguajes utilizados.

- **Capa de presentación (frontend):** esta capa es la que ve el usuario, presenta una interfaz gráfica del recurso solicitado y sirve para recoger su interacción. Suele estar situada en el cliente. La programación de esta capa se centra en el formato de la información enviada por el servidor y la captura de las acciones realizadas por el cliente.

- **Capa de negocio (backend):** es la capa que conoce y gestiona las funcionalidades que esperamos del sistema o aplicación web (lógica o reglas de negocio). Habitualmente es donde se reciben las peticiones del usuario y desde donde se envían las respuestas apropiadas tras el procesamiento de la información proporcionada por el cliente. Al contrario que la capa de presentación, la lógica de negocio puede ser programada tanto en el entorno cliente como en el entorno servidor.
- **Capa de persistencia (base de datos):** es la capa donde residen los datos y la encargada de acceder a los mismos. normalmente, está formada por uno o más gestores de bases de datos que realizan todo el proceso de administración de datos, reciben solicitudes de almacenamiento o recuperación de información desde la capa de negocio.

1.1.2 Tipos de arquitecturas cliente/servidor

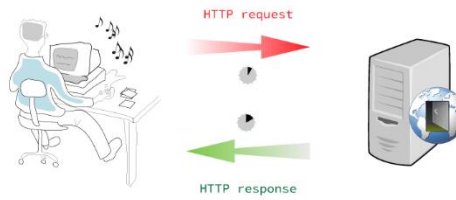
El diseño de cada una de estas capas influye a la hora de seleccionar uno u otro modelo de programación en la Web. Dichos modelos de programación cliente/servidor pueden clasificarse de este modo atendiendo a diferentes criterios. Así, es posible clasificarlos dependiendo del tamaño de los componentes, por la naturaleza del servicio ofrecido o según el reparto de funciones entre cliente y servidor:

- **Según el tamaño de los componentes:** esta primera clasificación hace referencia a qué elemento de la arquitectura web debe soportar más o menos carga de procesamiento. Se habla de configuraciones **Fat Client** (thin Server), donde el mayor peso de la aplicación se ejecuta en el cliente relegando al servidor a un mero administrador de datos; o **Fat Server** (thin client), donde la funcionalidad asociada al cliente está limitada a la presentación de la información enviada por el servidor.
- **Según la naturaleza del servicio ofrecido:** también es posible clasificar los entornos cliente/servidor en función de las capacidades ofrecidas por el servidor. De esta forma, podemos encontrar **servidores de ficheros**, donde el objetivo del cliente es el acceso a datos contenidos en ficheros; **servidores de bases de datos**, que se centran en la provisión y administración de sistemas gestores de bases de datos; **servidores de transacciones**, centrados en el concepto de transacción con el objetivo de que los flujos de información con el cliente se realicen en un solo mensaje solicitud/respuesta; **servidores de objetos**, cuya principal característica es la utilización de objetos intercomunicados, tanto en el cliente como en el servidor; o servidores web, que conforman la base del modelo World Wide Web y que está fundamentado en la existencia de clientes simples que se comunican con servidores web utilizando Http como protocolo de comunicación.
- **Reparto de funciones entre cliente y servidor:** las diferentes tecnologías web existentes permiten gestionar y distribuir las responsabilidades de cada una de las prestaciones funcionales entre el cliente y el servidor. Lo más habitual es tener una configuración cliente/servidor de dos o tres capas, dependiendo de si las capas de negocio y datos se agrupan (modelo en dos capas) o si se separan (modelo en tres capas). La separación en dos o tres capas la podemos ver, además, tanto desde el punto de vista del software como del hardware.

1.2 Páginas Web Estáticas y Dinámicas

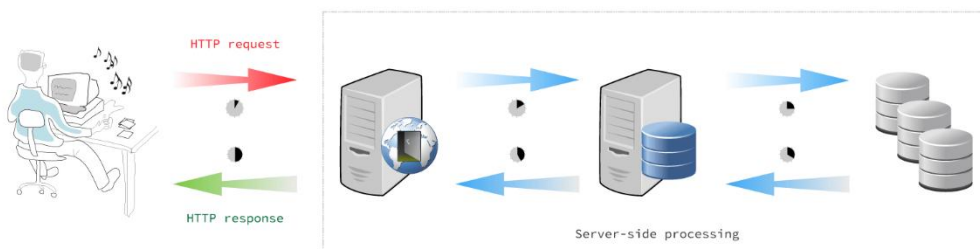
Static Website

Scheme A



Dynamic Website

Scheme B



Las páginas **estáticas** se encuentran almacenadas en su forma definitiva, tal y como se crearon, y su contenido no varía. Son útiles para mostrar una información concreta, y mostrarán esa misma información cada vez que se carguen. La única forma en que pueden cambiar es si un programador la modifica y actualiza su contenido.

En contraposición a las páginas web estáticas, existen las páginas web **dinámicas**. Estas páginas, como su nombre indica, se caracterizan porque su contenido cambia en función de diversas variables, como puede ser el navegador que estás usando, el usuario con el que te has identificado, o las acciones que has efectuado con anterioridad.

Dentro de las páginas web dinámicas, es muy importante distinguir dos tipos:

- Aquellas que incluyen código que ejecuta el navegador. En estas páginas el código ejecutable, normalmente en lenguaje JavaScript, se incluye dentro del HTML (o XHTML) y se descarga junto con la página. Cuando el navegador muestra la página en pantalla, ejecuta el código que la acompaña. Este código puede incorporar múltiples funcionalidades que pueden ir desde mostrar animaciones hasta cambiar totalmente la apariencia y el contenido de la página. En este módulo no vamos a ver JavaScript, salvo cuando éste se relaciona con la programación web del lado del servidor.
- Como ya sabes, hay muchas páginas en Internet que no tienen extensión .htm, .html o .xhtml. Muchas de estas páginas tienen extensiones como .php, .asp, .jsp, .cgi o .aspx. En éstas, el contenido que se descarga al navegador es similar al de una página web estática: HTML (o XHTML). Lo que cambia es la forma en que se obtiene ese contenido. Al contrario de lo que vimos hasta ahora, esas páginas no están almacenadas en el servidor; más concretamente, el contenido que se almacena no es el mismo que

después se envía al navegador. El HTML de estas páginas se forma como resultado de la ejecución de un programa, y esa ejecución tiene lugar en el servidor web (aunque no necesariamente por ese mismo servidor).

El esquema de funcionamiento de una página web dinámica es el siguiente:

Pasos:

1. El cliente web (navegador) de tu ordenador solicita a un servidor web una página web.
2. El servidor busca esa página y la recupera.
3. En el caso de que se trate de una página web dinámica, es decir, que su contenido deba ejecutarse para obtener el HTML que se devolverá, el servidor web contacta con el módulo responsable de ejecutar el código y se lo envía.
4. Como parte del proceso de ejecución, puede ser necesario obtener información de algún repositorio (Cualquier almacén de información digital, normalmente una base de datos), como por ejemplo consultar registros almacenados en una base de datos.
5. El resultado de la ejecución será una página en formato HTML, similar a cualquier otra página web no dinámica.
6. El servidor web envía el resultado obtenido al navegador, que la procesa y muestra en pantalla.

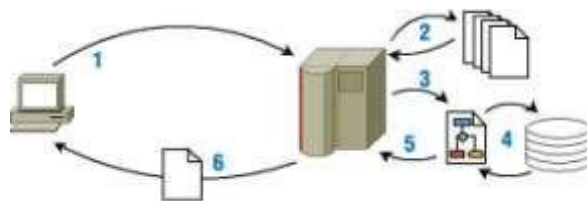


Ilustración 1-2. Funcionamiento Páginas Dinámicas

Este procedimiento tiene lugar constantemente mientras consultamos páginas web. Por ejemplo, cuando consultas tu correo en GMail, HotMail, Yahoo o cualquier otro servicio de correo vía web, lo primero que tienes que hacer es introducir tu nombre de usuario y contraseña. A continuación, lo más habitual es que el servidor te muestre una pantalla con la bandeja de entrada, en la que aparecen los mensajes recibidos en tu cuenta. Esta pantalla es un claro ejemplo de una página web dinámica.

Obviamente, el navegador no envía esa misma página a todos los usuarios, sino que la genera de forma dinámica en función de quién sea el usuario que se conecte. Para generarla ejecuta un programa que obtiene los datos de tu usuario (tus contactos, la lista de mensajes recibidos) y con ellos compone la página web que recibes desde el servidor web.

1.3 Lenguajes de Programación en Entorno Servidor

Se entiende por lenguaje de programación correspondiente a un entorno servidor a aquel cuyo código, bien sea como objeto precompilado o bien como código interpretado, es ejecutado por un software específico en el componente que actúa como servidor.

Existen múltiples alternativas a la hora de ejecutar código en el servidor. Una de ellas es utilizar lenguajes de scripting (SSI, LIVEWIRE, ASP, PHP, etc.), cuya característica principal

es que el código es interpretado y que se intercala con una plantilla de código HTML con la estructura básica de la página que se envía al cliente. Otra alternativa es el uso de enlaces a programas y componentes ejecutables (CGI, JSP, EJB, etc.), de tal forma que el código ejecutado por el servidor está almacenado en unidades precompiladas y ejecutadas de forma independiente, que generan las páginas enviadas al cliente. Otras técnicas de programación en el lado del servidor incluyen el uso de estrategias “híbridas” basadas en técnicas de respaldo (denominadas code-behind), como asP.net de Microsoft, en el que algunos elementos de código se intercalan con la lógica de presentación mientras que se mantiene la funcionalidad de dichos elementos en ficheros o librerías independientes en el servidor.

1.3.1 Lenguajes de Scripting

Se entiende por lenguaje de scripting (o embebido) aquel que se intercala con el código HTML que forma una aplicación web. la idea básica es crear una serie de plantillas HTML en las que se insertan instrucciones de programación en diferentes lenguajes que son ejecutadas por un servidor para determinar las partes dinámicas de las páginas web. Para que estas porciones de código sean ejecutadas, el agente que actúa de servidor (servidor web) debe tener instalado un módulo (scripting engine) que sea capaz de reconocer e interpretar el lenguaje en el que se programa dicho código.

Existen diversos lenguajes de scripting que pueden utilizarse para la creación de aplicaciones web dinámicas. Entre ellos destacan PHP, asP, Perl, Python y JsP:

- **PHP** (Hypertext Processor): es uno de los lenguajes más extendidos actualmente. Cuenta con una comunidad de desarrolladores muy importante debido a sus características de gratuidad, código abierto, la posibilidad de ser portado y ejecutado en diferentes plataformas, etc. se define como un lenguaje imperativo de tipos dinámicos, con la posibilidad de utilizar construcciones orientadas a objetos. Es soportado por la gran mayoría de servidores web actuales.
- **ASP** (Active Server Pages): se trata de una tecnología propietaria y de código cerrado para el lado del servidor de Microsoft. Su última versión es la 3.0 y data del año 2002, año a partir del cual se empezó a sustituir progresivamente por la versión asP.net. aunque puede ser ejecutado en otros servidores web, asP está diseñado especialmente para ser utilizado con IIS (Internet Information Server). el lenguaje utilizado con ASP es visual basic en su versión de scripting (vbscript), aunque también pueden usarse otros (Jscript).
- **Perl**: fue inicialmente concebido como un lenguaje de manipulación de cadenas de caracteres basado en un estilo de programación por bloques, como C o aWK. Perl es un lenguaje imperativo, con variables, expresiones, asignaciones, bloques de código delimitados por llaves, estructuras de control y subrutinas actualmente orientado a la generación dinámica de páginas web. Fue uno de los primeros lenguajes en ser utilizados para la programación web (para crear aplicaciones CGI). es de código abierto y cuenta con una comunidad bastante numerosa de seguidores.
- **Python**: es un lenguaje de scripting independiente de plataforma y orientado a objetos, preparado para realizar cualquier tipo de programas, desde aplicaciones Windows a servidores de red, o incluso páginas web. es un lenguaje interpretado, lo que significa que no necesitamos compilar el código fuente para poder ejecutarlo. esto

ofrece ventajas, como la rapidez de desarrollo, e inconvenientes, como una menor velocidad. en los últimos años el lenguaje se ha hecho muy popular gracias a la cantidad de librerías, tipos de datos y funciones incorporadas en el propio lenguaje o la sencillez y velocidad con la que se crean los programas. además, Python es un lenguaje gratuito, es decir, no necesita de ningún software de pago adicional para su ejecución incluso para propósitos empresariales.

- **JSP** (Java Server Pages): el funcionamiento de Java como lenguaje de script es similar al de PHP o asP: se trata de porciones de código Java intercalado con código HTML estático. sin embargo, la forma de interpretarlo es diferente. Una vez que el módulo encargado de ejecutar la página JsP llega a una porción de código Java, lo transforma a un Servlet (porción de código Java cargado en la memoria de un servidor web) y lo ejecuta obteniendo el código que ha de enviarse al cliente. la diferencia con otros lenguajes es que el servlet queda cargado en memoria por si llegara otra petición a la misma página JsP, con lo que el rendimiento se ve mejorado notablemente.

Existen muchos otros lenguajes de scripting en el lado del servidor enfocados a entornos más específicos y con características particulares, como ColdFusion Markup language, lua, ruby, sMx, lasso, WebDna o Progress Webspeed.

1.3.2 Aplicaciones CGI y Derivados

Una de las formas más simples de generar páginas dinámicas es delegar la creación de las mismas a un programa externo, que reciba ciertos parámetros de entrada y devuelva como resultado el contenido que debe visualizar el cliente. El estándar CGi (Common Gateway Interface), especificado en la RFC3875, define precisamente este comportamiento, estableciendo los vínculos que hay que establecer con una aplicación independiente o fichero ejecutable. Puesto que el programa externo no depende del código a generar, el lenguaje elegido puede ser cualquiera (como C o C++). La localización del script o fragmento de programa a ejecutar se indica en la Url que forma la petición HTTP del cliente.

Como hemos dicho, CGi es la forma más simple de publicar contenido dinámico en la Web; sin embargo, esta forma de crear aplicaciones web presenta ciertas desventajas, siendo la principal de ellas el escaso rendimiento a la hora de responder a múltiples peticiones CGi simultáneamente puesto que para cada una se tiene que crear un proceso nuevo en el servidor con los costes de memoria y uso de procesador que conlleva. respecto a este problema han surgido algunas alternativas como son el uso de extensiones para cada uno de los lenguajes en los que se programe el CGi pero como parte integrante del servidor web (módulos nsaPi o isaPi por ejemplo), o el uso de FastCGI como forma de crear un único proceso que atienda a todas las peticiones CGi.

Otra de las alternativas que se han propuesto como solución a los problemas de CGi es el uso de plataformas independientes de ejecución de código. El principio de estas plataformas es equivalente al uso de extensiones para servidores web. Es el caso de la programación con Java en el lado del servidor. En este contexto, la arquitectura de un servidor que es capaz de ejecutar programas Java (Servlets, JavaBeans, etc.) está formada por el propio servidor web en sí mismo y la Máquina virtual de Java (JvM), que es capaz de ejecutar un programa especial Java (llamado Servlet container) encargado de gestionar datos de sesión y Servlets.

Otra de las alternativas que un desarrollador tiene a su disposición cuando programa con Java es usar los denominados **Enterprise Java Beans (eJb)**, que son componentes programados en Java que implementan la lógica de negocio de la aplicación utilizando las características de la plataforma J2ee. ASu especificación detalla cómo los servidores de aplicaciones proveen objetos desde el lado del servidor que son, precisamente, los eJb. el uso de eJb está especialmente indicado en los casos en los que se requiera mantener la transaccionalidad de las peticiones hechas por un cliente en un entorno web.

1.3.3 Aplicaciones Híbridas de código repartido (Code-Behind)

Como alternativa a los lenguajes de scripting (interpretados) y a las aplicaciones CGI y derivadas (ejecutadas por un programa o módulo independiente del servidor web), en los últimos años ha surgido una tecnología intermedia que podríamos denominar como “híbrida”. La solución más representativa es la plataforma de Microsoft .net Framework a través de ASP.net.

ASP.net es una tecnología totalmente orientada a objetos que puede ser escrita en cualquier lenguaje soportado por el entorno .net Framework (vb.net; C# y Jscript.net). Desde el punto de vista del rendimiento, la aplicación se precompila en una sola vez al lenguaje nativo y, luego, en cada petición tiene una compilación Just in Time, es decir, se compila desde el código nativo, lo que permite mucho mejor rendimiento.

Las páginas de ASP.net, conocidas oficialmente como Web Forms (formularios web), son el principal medio de construcción para el desarrollo de aplicaciones web desde el punto de vista de Microsoft. Los formularios web están contenidos en archivos con una extensión ASPX que son los que el cliente solicita a través de una URL al servidor. Estos ficheros ASPX contienen código HTML o estático y también etiquetas propias de la plataforma .net. Estas etiquetas definen Controles Web que se procesan del lado del servidor y Controles de Usuario donde los desarrolladores colocan todo el código estático y dinámico requerido por la página web.

Adicionalmente, el código dinámico que se ejecuta en el servidor puede ser colocado en una página dentro de un bloque “<% -- código dinámico -- %>” que es muy similar a otras tecnologías de scripting como PHP, JSP y ASP. Esta circunstancia es otra de las razones por las que se considera la programación con ASP.net como “híbrida”. En algunos entornos de alto rendimiento y seguridad esta práctica es, generalmente, desaconsejada excepto para propósitos de enlace de datos, pues requiere más llamadas cuando se genera la página.

Una de las mayores ventajas de la arquitectura .Net es que incluye todo lo necesario para el desarrollo y el despliegue de aplicaciones. Por ejemplo, tiene su propio entorno de desarrollo, Visual Studio, aunque hay otras opciones disponibles. La mayor desventaja es que se trata de una plataforma comercial de código propietario.

1.3.4 Ventajas e inconvenientes de cada tipo de lenguaje

Cada una de estas formas de ejecución del código por el servidor web tiene sus ventajas e inconvenientes.

- Los lenguajes de **scripting** tienen la ventaja de que no es necesario traducir el código fuente original para ser ejecutados, lo que aumenta su portabilidad. Si se necesita

realizar alguna modificación a un programa, se puede hacer en el momento. Por el contrario el proceso de interpretación ofrece un peor rendimiento que las otras alternativas.

- Los lenguajes compilados a código nativo o aplicaciones **CGI** son los de mayor velocidad de ejecución, pero tienen problemas en lo relativo a su integración con el servidor web. Son programas de propósito general que no están pensados para ejecutarse en el entorno de un servidor web. Por ejemplo, no se reutilizan los procesos para atender a varias peticiones: por cada petición que se haga al servidor web, se debe ejecutar un nuevo proceso. Además los programas no son portables entre distintas plataformas.
- Los lenguajes compilados a código intermedio o **aplicaciones híbridas** ofrecen un equilibrio entre las dos opciones anteriores. Su rendimiento es muy bueno y pueden portarse entre distintas plataformas en las que exista una implementación de la arquitectura (como un contenedor de servlets o un servidor de aplicaciones Java EE).

1.4 Integración con los Servidores Web

Hasta ahora hemos comentado que para el desarrollo de aplicaciones web en entornos cliente/servidor pueden utilizarse diferentes modelos, tecnologías y lenguajes. En cualquier caso, en muchas ocasiones, la posibilidad de responder a una petición hecha por un cliente depende de las capacidades que tenga el servidor web y los módulos o extensiones que tenga instalados.

Empezando por la funcionalidad básica de un servidor web, su cometido principal es proveer de contenido estático a un cliente que ha realizado una petición a través de un navegador. Para ello, carga un archivo y lo sirve a través de la red al navegador del solicitante. Este es el funcionamiento habitual cuando lo que se implementa en el servidor es una aplicación web estática (HTML) o dinámica (DHTML), es decir, que el servidor se convierte en un instrumento que proporciona un lugar para guardar y administrar los recursos HTML, que pueden ser accesibles por los usuarios de la red a través de navegadores.

Para que el servidor pueda entender la petición del cliente, éste tiene que realizar una petición “formal”, es decir, la petición tiene que constar de unos elementos concretos y especificados en un orden determinado. Las direcciones de las peticiones suelen ser de tipo URL (Localizador Uniforme de Recurso), que contiene una dirección, la referencia a un cierto protocolo (HTTP, FTP, etc.) y la descripción de un recurso concreto en forma de ruta al objeto que queremos descargar. Opcionalmente podemos incluir un campo adicional (puerto) indicando el número de puerto por el que el servidor está escuchando las peticiones del cliente.

Con respecto a las peticiones de los clientes es importante destacar que existen distintos modos o métodos para intercambiar información entre cliente y servidor y que deberemos tenerlo en cuenta en el desarrollo de aplicaciones web en el entorno del servidor.

- **Método GET:** es un método de invocación en el que el cliente le solicita al servidor web que le devuelva la información identificada en la propia URL. Lo más común es que las peticiones se refieran a un documento HTML o a una imagen, aunque también se puede referir a un programa de base de datos. En tal caso, el

servidor ejecuta ese programa y le devuelve al cliente el resultado generado tras esa petición.

- **Método POST:** El método Post se usa habitualmente para enviar información a un servidor web. Estos casos suelen darse al enviar el contenido de un formulario de autenticación, así como entradas de datos o especificar parámetros para algún tipo de componente ejecutado en el servidor.

Como ya se ha comentado anteriormente, la potencia de los servidores web aparece en el momento en que queremos desarrollar aplicaciones web dinámicas y con una cierta complejidad, en ese caso, podemos utilizar varias tecnologías implementadas en módulos específicos en el servidor para aumentar su potencia más allá de su capacidad de entregar páginas HTML. Dichas tecnologías suelen instalarse como extensiones en el software del servidor e incluyen soporte para scripts CGI, proporcionan seguridad ssl, permiten la ejecución de scripts, interactúan con la máquina virtual de Java, etc.

1.5 Herramientas de Programación

La selección de un entorno de desarrollo para la programación web debe tener en cuenta los diferentes escenarios, modelos y configuraciones que intervienen en este contexto. las aplicaciones web están formadas por un conjunto de páginas HTML, scripts de lenguajes web, programas escritos en diferentes lenguajes, bases de datos y documentos de diferentes formatos y pueden residir en varias ubicaciones o sitios web, es decir, en diferentes servidores.

El proceso de desarrollo, sin embargo, no tiene por qué realizarse en el mismo equipo en el que finalmente se despliegue y ejecute la aplicación web que se está desarrollando. el mismo razonamiento es aplicable al proceso de pruebas en el que se simulen las peticiones al servidor web desde un cliente determinado. Para el proceso de desarrollo deberemos tener en cuenta, en el caso del servidor, el conjunto de tecnologías y lenguajes seleccionados, el tipo de servidor, los módulos y extensiones que tiene configurado, los permisos de acceso y ejecución o la localización de los datos utilizados por la aplicación web. Desde el punto de vista del cliente habrá que tener en cuenta, entre otros, la versión y tipo de navegador, la resolución gráfica o el sistema de permisos del navegador en el equipo cliente.

A la hora de seleccionar las herramientas que un desarrollador tiene a su disposición es necesario hacer una primera clasificación de los instrumentos involucrados en función de sus capacidades:

- **Navegadores.** son las aplicaciones que se ejecutan en el entorno del cliente y que permiten visualizar los documentos escritos en lenguaje HTML y capturar las interacciones del usuario.
- **Editores de documentos.** este grupo está formado por editores de texto que permiten escribir código HTML directamente, sin ninguna ayuda ni facilidad adicional.
- **Entornos de programación.** son entornos integrados que nos permiten editar, compilar y ejecutar los programas generados a partir de diferentes lenguajes usados en el desarrollo de las aplicaciones web.
- **Herramientas de tratamiento de imágenes.** la mayoría de las páginas web muestran contenido gráfico de una u otra manera, por ello, es necesario el uso de este tipo de

herramientas para adecuar las características de las imágenes a su transmisión por la red y su posterior visualización.

- **Herramientas para la creación y administración de bases de datos.** se trata de herramientas para la carga de datos y el mantenimiento posterior de los datos almacenados.

En el ámbito del desarrollo de aplicaciones web las herramientas más importantes son los llamados entornos de programación, que son programas, aplicaciones o simples utilidades destinadas a la programación web, haciendo mucho más fácil para el programador su tarea. Desde el punto de vista de las herramientas que se pueden utilizar para la construcción de programas o scripts, podemos emplear una gran variedad de utilidades que van desde simples editores de texto hasta los entornos integrados que facilitan enormemente la construcción de aplicaciones web en lenguajes específicos.

1.5.1 Marcadores de Texto

Los marcadores de texto, o text markers, son simples editores de texto, aunque dirigidos al ámbito de la programación. Al escribir el programa, y dependiendo del lenguaje de programación utilizado, el editor de texto nos ayuda a identificar mejor la sintaxis del lenguaje, cambiando de color las etiquetas, realizando tabulaciones en el texto, etc. algunas de ellas son:

- **Arachnophilia.** es un robusto editor de texto, ideal para programar sitios web en diferentes lenguajes. soporta HTML, Javascript, C++, CGi, Perl, Java, entre otros. también incluye un cliente inteligente de FtP para subir automáticamente los archivos modificados.
- **Notepad++.** es un editor gratuito de código fuente. soporta varios lenguajes de programación y se ejecuta en Windows.
- **UltraEdit.** Completo editor de texto para programación. soporta múltiples formatos con colores configurados para cada lenguaje. es uno de los múltiples editores de pago que podemos encontrar en el mercado.
- **Sublimetext.** Editor de texto avanzado especialmente diseñado para desarrolladores que realmente sale de lo común y se destaca por sus funcionalidades e interfaz de usuario. Básicamente, tiene muchas de las funcionalidades que podemos encontrar en otros editores avanzados como NOTEPAD++ o Lopedit, pero si lo analizamos profundamente encontraremos varias ventajas que realmente lo posicionan entre los mejores editores de texto.
- **Atom.** Editor de texto creado por Github. Github es uno de los mejores repositorios para software libre de la actualidad, así que es normal tener altas expectativas con su editor de texto; pero pese a eso, Atom las cumple de manera sobrada. Ha sido el editor de texto que ha conseguido hacerse un hueco entre los programadores y los partidarios del software libre. Ha sido creado desde el inicio basándose en los principios de libertad y personalización, y de hecho el proyecto se define a si mismo como el editor de texto “hackeable” del siglo XXI.
- **Visual Studio Code.** Visual Studio Code es un editor de código fuente desarrollado por Microsoft para Windows , Linux y macOS. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y

refactorización de código. También es personalizable, por lo que los usuarios pueden cambiar el tema del editor, los atajos de teclado y las preferencias. Es gratuito y de código abierto, aunque la descarga oficial está bajo software propietario. Su popularidad y uso avanza día a día, llegando a ser hoy por hoy uno de los más utilizados entre los desarrolladores PHP.

1.5.2 Herramientas Genéricas

Dentro de los entornos de programación existen herramientas que ofrecen funcionalidades avanzadas aparte del reconocimiento de la sintaxis del lenguaje. estas aplicaciones están mucho más desarrolladas que los marcadores de texto y ofrecen capacidades tales como la sugerencia de estructuras o funciones predeterminadas o la posibilidad de validar la corrección del código escrito. a continuación describimos algunas de estas herramientas a modo de ejemplo:

- **Microsoft frontPage.** es una herramienta de construcción y edición de páginas web para el sistema operativo Windows. si bien tuvo bastante éxito hace unos años, actualmente muchos desarrolladores consideran que el código HTML generado por esta aplicación es un poco descuidado y muchas veces reiterativo, especialmente en versiones antiguas. Forma parte del paquete ofimático Microsoft office y es de pago.
- **Eclipse.** se trata de un entorno de programación gratuito escrito en Java que permite desarrollar aplicaciones en varios lenguajes (C, Java, PHP...). Una de sus principales características es que permite la extensión de sus funcionalidades mediante la instalación de múltiples módulos para diferentes propósitos específicos.
- **Dreamweaver.** es un editor de código HTML profesional de pago para el diseño visual y la administración de sitios y páginas web. también incluye numerosas herramientas y funciones de edición de código: referencias HTML, Css y Javascript, un depurador Javascript y editores de código (la vista de código y el inspector de código) que permiten editar Javascript, xML y otros documentos de texto directamente en Dreamweaver.

1.5.3 Herramientas Específicas

En muchas ocasiones, la utilización de un lenguaje de programación determinado exige que el desarrollador tenga instalada una plataforma de desarrollo concreta. tal es el caso de la programación en Java que requiere la instalación de la Máquina virtual de Java (JvM) o de el entorno .net Framework de Microsoft, necesaria para la programación con asP.net. las herramientas que mostramos a continuación se encuentran dentro de la categoría de herramientas específicas para una tecnología concreta.

- **Microsoft Visual Studio.** Es el entorno de desarrollo más conocido para el diseño de aplicaciones en los sistemas operativos de Microsoft. está destinado al desarrollo de aplicaciones web con lenguajes como visual basic o C++ y permite la publicación de la aplicación web que se está desarrollando directamente desde su interfaz.
- **NetBeans IDE.** Es una aplicación de código abierto diseñada para el desarrollo de aplicaciones fácilmente portables entre distintas plataformas haciendo uso de la tecnología Java. se trata de un entorno de desarrollo gratuito optimizado para el desarrollo de aplicaciones con Java.

1.6 Servicios Web

Para poder desarrollar nuestras aplicaciones necesitamos usar un servidor web libre que suministre los siguientes servicios web básicos:

- Servidor Web
- Servidor de Base de Datos
- Intérprete para los lenguajes de scripting como PHP

Para conseguir todos estos servicios en nuestro equipo de desarrollo, una de las mejores herramienta existente en el mercado es **XAMPP**.

1.6.1 Plataforma XAMPP

XAMPP es un servidor independiente de plataforma, software libre. Consiste principalmente en el sistema de gestión de bases de datos **MySQL**, el servidor web **Apache** y los intérpretes para lenguajes de script: **PHP** y Perl. El nombre proviene del acrónimo de X (para cualquiera de los diferentes sistemas operativos), Apache, MySQL, PHP, Perl. Desde la versión "5.6.15", XAMPP cambió la base de datos de MySQL A MariaDB. El cual es un fork de MySQL con licencia GPL.

El programa se distribuye bajo la licencia GNU y actúa como un servidor web libre, fácil de usar y capaz de interpretar páginas dinámicas. Actualmente XAMPP está disponible para Microsoft Windows, GNU/Linux, Solaris y Mac OS X.

1.6.2 MySQL Workbench

MySQL Workbench es una herramienta visual de diseño de bases de datos que integra desarrollo de software, Administración de bases de datos, diseño de bases de datos, creación y mantenimiento para el sistema de base de datos MySQL. Se aconseja su instalación para la gestión de las bases de datos en un entorno fácil y amigable, en caso contrario debemos optar al PHPMyadmin de XAMPP.